Deep Reinforcement Learning Assignment

Spring 2026

MSc Cognitive Science and Artificial Intelligence
Tilburg University

Welcome to the assignment for the Deep Reinforcement Learning course, Spring 2026. This assignment is composed of exercises and mini-projects, organized in two parts.

You are asked to submit a report outlining your development of the exercises in the assignment, with particular focus on motivating your choices and discussing your results.

The deadline for submitting your report is **June 16th**. You are advised to start acquainting yourself with the assignment early on; the exercises should not be excessively difficult, but it may take some time to get used to the instructions and the provided code.

1 [15 pts] Part 1: Multi-Armed Bandits

Consider the k-armed bandit problem, where an agent must choose among k arms (slot machines) at each timestep. Each arm i yields a stochastic reward drawn from a Gaussian distribution $\mathcal{N}(\mu_i, 1)$, where μ_i is the unknown true mean of arm i. The agent's goal is to maximize its cumulative reward (or equivalently, minimize its cumulative *regret*).
We provide starter code (`part1_bandits/bandits.py`) with:

- A `BanditEnvironment` class implementing a k-armed Gaussian bandit
- A complete `EpsilonGreedy` agent ($\epsilon = 0.1$) for reference
- Functions for running experiments, computing cumulative regret, and plotting

Your tasks:

- **1.1 (5 pts):** Implement the **UCB1** (Upper Confidence Bound) agent. Recall the UCB1 arm selection rule [1]:

$$A_t = \arg\max_a \left[Q(a) + c\sqrt{\frac{\ln t}{N(a)}} \right] \tag{1.1}$$

where $Q(a)$ is the estimated value of arm a, $N(a)$ is the number of times arm a has been pulled, t is the current timestep, and c is an exploration parameter. Use $c = 2$ as a starting point.

- **1.2 (5 pts):** Implement a **Thompson Sampling** agent [2]. Thompson Sampling maintains a posterior distribution over each arm's mean reward and selects arms by sampling from these posteriors.
  For our Gaussian bandit (known variance $\sigma^2 = 1$) with a $\mathcal{N}(0, 1)$ prior on each arm's mean, the posterior after n pulls with observed sample mean $\bar{x}$ is:

$$\mu_a \mid \text{data} \sim \mathcal{N}\left(\frac{n \cdot \bar{x}}{n+1},\ \frac{1}{n+1}\right) \tag{1.2}$$

  (This follows from conjugate Bayesian updating of a Gaussian prior with a Gaussian likelihood.)
  The algorithm at each timestep: (1) for each arm a, sample θ_a from the posterior; (2) select the arm with the highest θ_a.
- **1.3 (5 pts):** Run all three agents (Epsilon-Greedy, UCB, Thompson Sampling) on a 10-armed bandit for 1000 steps, averaged over at least 200 independent runs (different bandit instances). Plot the cumulative regret curves for all agents on the same figure.
  Discuss: Which agent accumulates the least regret? Under what conditions (e.g., number of arms, time horizon) would you expect the ranking to change? How does the exploration strategy of UCB differ fundamentally from that of Epsilon-Greedy?

Requirements: Only `numpy` and `matplotlib` are needed. No GPU required.

2 [60 pts] Part 2: Reinforcement Learning from Human Feedback (RLHF)

In Part 2, you will apply Reinforcement Learning from Human Feedback (RLHF) to fine-tune a small language model. RLHF is the technique used to align large language models like ChatGPT and Claude with human preferences [3].

2.1 Background

The standard RLHF pipeline consists of three stages:

1. **Supervised Fine-Tuning (SFT):** A pre-trained language model is fine-tuned on a dataset of demonstrations (e.g., high-quality text).
2. **Reward Model Training:** A reward model is trained on pairs of outputs labeled by human preferences ("output A is better than output B").
3. **PPO Fine-Tuning:** The SFT model is further fine-tuned using PPO [4], with the reward model providing the reward signal. A KL penalty term prevents the model from deviating too far from the SFT model (to avoid overfitting and maintain high text quality).

In this assignment, we simplify this pipeline:

- For **SFT**: We use an existing GPT-2 model already fine-tuned on IMDb movie reviews (`lvwerra/gpt2-imdb`). This model generates movie review text — both positive and negative.

- For the **reward model**: We use a pre-trained sentiment classifier (`lvwerra/distilbert-imdb`) that returns the probability of a text being positive. No training needed.
- For **PPO fine-tuning**: We use the TRL library [5] which implements PPO for language models. The goal is to steer the model toward generating *positive* movie reviews.

The PPO objective (for language models) is:

$$\max_{\theta} \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[R(y) - \beta\, \mathrm{KL}(\pi_\theta(\cdot|x) \,\|\, \pi_{\mathrm{ref}}(\cdot|x))\right] \tag{2.1}$$

where $R(y)$ is the reward (sentiment score), π_θ is the current policy, π_{ref} is the reference (SFT) model, and β is the **KL penalty coefficient**. The KL term prevents the policy from *overfitting* to the reward model: without it, the model can drift arbitrarily far from the pre-trained model and find degenerate outputs that score highly on the reward model but are not meaningful text (a phenomenon known as *reward hacking*). By keeping the policy "close" to the pre-trained reference model (in the sense of KL divergence), we ensure that the model retains the language capabilities it learned during pre-training while steering its outputs toward higher reward.

2.2 Setup

We provide a Colab notebook (`part2_rlhf/rlhf_sentiment.ipynb`) with most of the pipeline already implemented. The notebook uses:

- `trl==0.9.6` — TRL library for PPO training of language models
- `transformers` — HuggingFace Transformers for model loading
- `datasets` — HuggingFace Datasets for loading IMDb data

IMPORTANT: This notebook requires a **GPU**. We recommend running it on Google Colab with a T4 GPU runtime. Each training run takes approximately 30–60 minutes.
If you run your code on Colab, please look up how the filesystem is managed, so that you can retrieve the output files (anything you wish to save, output plots, etc.). You can either interact with the local (temporary) drive associated with your Colab notebook, or manually mount your own GoogleDrive:

```
import os
from google.colab import drive
drive.mount('/mnt/gdrive')

output_dir = "/mnt/gdrive/My Drive/Colab Notebooks/rlhf_results"
os.makedirs(output_dir, exist_ok=True)
```

2.3 Your tasks

- **2a (10/60 pts):** Run the provided RLHF pipeline with the default KL coefficient ($\beta = 0.2$). The notebook walks you through each step: loading the model, computing rewards via the sentiment classifier, and running the PPO training loop. Generate text samples *before* and *after* RLHF. Show concrete examples in your report and discuss how the model's outputs changed. Are the post-training outputs all positive? Are they coherent?
- **2b (50/60 pts):** Investigate the effect of the KL penalty coefficient β.
  Run the training pipeline for **multiple** values of β (at least 3–5 different values, covering a wide range — e.g., from very small to large). The notebook provides a helper function to re-run the experiment with different KL coefficients.

For each value of β, you should track the reward and KL divergence during training, as well as the quality of the generated text at the end of training.

In your report:

- Present plots and/or text samples that illustrate how the KL coefficient affects training dynamics, final performance, and text quality.
- Identify and discuss **reward hacking**: for which β value(s) does the model produce text that achieves high sentiment scores but is degenerate or meaningless? Show examples and explain *why* this happens.
- Discuss the **reward–KL tradeoff**: why can't we achieve both very high reward and very low KL divergence? What does this imply for real-world RLHF systems (e.g., ChatGPT, Claude)?

References

[1] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2-3, pp. 235–256, 2002.

[2] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.

[3] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.

[5] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang, "Trl: Transformer reinforcement learning." `https://github.com/huggingface/trl`, 2022.